

Your Agent Escaped Without Escaping the Sandbox

The agent changes the answers.
The check goes green.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–20 are for questions.

1. The ticket

00:00-00:45

Read the ticket. The four locks. Then: it ships.

Picture a Friday afternoon. You give your coding agent one ticket. Lock down the admin page. Anyone without a login must get 401.

And you do everything right. The agent works in a sandbox. It has no network. The checker that grades its work is read-only, so the agent cannot touch it. And your pipeline ships only when that checker goes green.

The agent works for a while. The check goes green. It ships.

2. Green, and open

00:45-01:45

Say green. Pause. Then 200. Let it sit.

Then someone opens the admin page without logging in. 200. Anyone can walk in.

The door you asked the agent to close is still open, and every check was green. And here is the strange part. Nothing escaped the sandbox. No network call, no broken container, no stolen credential. If you have spent this year building sandboxes for your agents, this is the part the sandbox cannot see.

I'm Yossi Eliaz. I work on build systems at Incredibuild, and our whole industry runs on one assumption: green means good. For the next few minutes, let's find out what green meant here.

3. We built that pipeline

01:45-02:45

Four facts, calmly. Then: three suspects.

So we built that pipeline ourselves, small enough to check every byte. The program is a few lines of command-line code that model the admin page. No web server. It answers five cases: no login, a real admin, an ordinary user, the public page, and an expired login. The agent is a script, not a model, so every step is deterministic. Hold on to that objection. We will come back to it. Everything runs in a Linux sandbox, and every file that matters is hashed.

We stopped at the green check. Your pipeline would not have.

So let's investigate. Three suspects.

4. Three suspects

02:45-03:45

Clear the code. Clear the checker. Then: something else changed.

Suspect one: the code. The ticket was to fix it. Did the agent change it at all? No. The program's hash is identical before and after, and it still prints 200 for every request.

Suspect two: the checker. And the agent did try. It tried to overwrite the checker with one that just prints PASS. The read-only mount said DENIED, and the checker's hash never changed. Before the agent's step, this same checker, launched the same way, said FAIL.

So the code did not change. The checker did not change. And FAIL still became PASS. Something else changed.

5. Suspect three: the answer key

03:45-05:00

Name the file. Read the diff. The hashes. Click. Land the line.

Suspect three is a file nobody thinks about. The checker does not know the right answers by itself. It reads them from a file in the workspace, `expected.json`. The answer key.

And the agent is allowed to write the workspace. So it changed one line. No login: expected 401. Now: expected 200. It did the same for the two other cases that expected an error. Now every expected answer is 200, which is exactly what the broken code prints.

We had perfect hashes of every file here. They told us exactly what changed. They did not stop it.

It did not make the code pass the test. It made the test pass the code.

6. "A real model wouldn't."

05:00-06:15

The objection. METR. Click at: It overwrote the timing function. Then the stopwatch. Say METR as meter.

Now the objection you have been holding. Our agent is a script. It did exactly what we wrote. Would a real model do this?

In June 2025, the AI evaluation nonprofit METR published what frontier models did on tasks they were scored on. OpenAI's o3 was asked to make a program run faster. It changed something else. It overwrote the timing function that measured it, so a real second read as a millisecond. On another task, it patched the evaluator so that every submission passed. Nobody asked it to. In a separate test, telling it not to cheat barely changed how often it planned to.

It changed the stopwatch.

7. Part of the judge was inside the sandbox

06:15-07:15

Why did the door stay open? Walk the picture. Point at expected.json. End on the judge.

So why did the door stay open? Here is the whole thing in one picture. This dashed box is the agent's sandbox. The agent is inside, with its code. The checker is outside, read-only, exactly as we built it. But the checker reaches into the workspace to read its answer key.

o3's code ran inside the program that scored it. Our agent could not reach its checker, so it rewrote what the checker reads. Same move: change the judge, not the work.

The judge is not the checker. The judge is the checker plus everything it reads. And part of your judge was inside the sandbox.

8. Where is your answer key?

07:15-09:00

Show of hands, yours up too: writes tests, then keep it up if sandboxed. Look around. Then the procurement agent. Then: the permission is the work.

Now look for the answer key in your own systems. Quick show of hands, and I will raise mine too. Who has a coding agent that is allowed to write tests? Keep it up if that agent runs in a sandbox. Look around. Those hands did everything right. Every one of them still has an answer key the agent can write.

Outside code, picture a procurement agent. Part of its job is keeping the approved vendor list current. It is asked to pay an invoice, and the payment check blocks it: the vendor is not on the list. So the agent adds the vendor. The payment check goes green. Nothing escaped. The same shape shows up wherever a check reads something the agent can write: the policy a guardrail checks, the notes a monitor reads, the summary an LLM judge grades.

Some of these you can lock, and you should. For the rest, you cannot just take the pen away. Writing tests is the coding agent's job. Keeping that list is the procurement agent's job. The permission is the work.

9. The rule

09:00-10:00

The rule, slowly. Then back to the ticket: REJECT, the door never opens, ACCEPT.

So here is the rule. The agent may propose what counts as correct. It must never be the last writer of what judges it. Its new tests, its new vendor, are proposals. Something the agent cannot write decides whether they count: a reviewer, or a controller that owns the criteria.

Back to our ticket. In the same recording, we also ran it that way: the answer key belongs to the controller, where the agent cannot write. Same broken code: REJECT. The door never opens. The agent's edit is still sitting in the workspace. It just no longer counts. Then the corrected code: ACCEPT. Now the ticket is actually done.

10. It was one of four

10:00-11:00

The answer key was one of four. One breath per row: the escape, then the boundary. End on the job still finishing.

The answer key was the quietest of four non-escape escapes, and we recorded all four in the same Linux lab. An inherited runner token read another project. A short-lived, task-scoped token got 403. A writable mount changed the next job's files. A read-only, scoped mount blocked the same write. An allowed upload host carried synthetic private data to another account. With no direct network, and a broker that builds one fixed report, nothing was delivered. And the answer key: controller-owned criteria, and a history that shows any rewrite.

Four boundaries, each at the layer that can actually enforce it. And in every case, the real job still finished.

11. One question for Monday

11:00-12:15

Before your next ticket ships. The question. Pause. Three moves.

Before your next ticket ships, one question. What does your judge read that your agent can write?

Three moves. First, list it. Every file, config, environment variable and service your checks read. Cross off the code under test. Anything left that the agent can write is your finding. Second, move the copy that counts to where the agent cannot write. When the agent has to change it, the change is a proposal, and a reviewer decides, not the green check. Third, keep the code under test out of the judge's process, so it can only answer, not grade. That is the route o3 used.

None of this needs model telemetry. It needs an inventory and an owner.

12. The door stayed open

12:15-13:00

Repository first. Then the ticket. Three short lines. The title. Thank you. Leave it up.

The code and the recordings are in the repository on the screen. Every hash you saw is in there, and you can rerun all of it.

Now remember the ticket. Lock down the admin page.

The sandbox held. The checker held. The door stayed open. The agent never had to leave the sandbox. It only had to change what counted as correct.

Your agent escaped without escaping the sandbox. Thank you.

Questions

13. The five policy cases

Questions

Use only in questions.

The handler takes a route and an authentication label and prints an integer. The runner checks these five. The repaired handler matches all five. This is the model of the check that promotes into the prod sandbox.

14. Runner token and the dev disk

Questions

Use only in questions.

The token case is loopback HTTP. The broad fixture stands in for the runner secret. The task fixture stands in for a job-scoped token. The mount case watches the host file after an ordinary write from the dev sandbox. This job's output stays writable.

15. Allowed host, wrong account

Questions

Use only in questions.

The fixture service permits both account paths, the way one allowed deploy host holds more than one operation. The delivery record shows the bytes arrived. The repair moves the POST onto the runner and removes the dev sandbox's direct route.

16. An earlier promote stored a marker

Questions

Use only in questions.

The archived case mutates a controller-side object, keeps a stale digest, and the old gate records a publication marker. The current promote derives the digest from immutable bytes and stores those bytes. Reaching the old flaw takes access to the controller object.

17. What was recorded

Questions

Use only in questions.

The records bind their source. The Linux recordings use bubblewrap on an Ubuntu machine. The worker under test is a script. Publication in the recording is the controller storing the exact approved bytes in a local database, a stand-in for a registry or a merge. Real-repository, multi-agent and snapshot effects are inferences from the answer-key dependency, not measurements. The timings are a delivery budget.

18. Reproduce the factory

Questions

Use only in questions.

Run the fixtures on disposable Linux with bubblewrap. The deck reads those recordings. Poisoned pipeline execution, OWASP CICD-SEC-4, is the public name for a CI job made to run commands an attacker changed in the repository. Ours is its data-only neighbor: the checker ran unchanged and obeyed a file the worker rewrote. Anthropic's reward-hacking study is prior motivation and its measurements stay with that study. Reproducing the talk needs no GitHub token.

19. The paired run

Questions

Use only in questions.

The paired run behind slides 4 and 5. Same checker bytes, same launch, same program. The honest answer key gives exit 1 with FAIL. The edited answer key gives exit 0 with PASS. The answer key is the only variable. Every prefix is a full SHA-256 in the isolated recording.

20. One unused approval

Questions

Use only in questions.

Four attempts on one approval, in the recorded order. Bad bytes while the approval is still unused: denied. The approved bytes to another destination: denied. The approved bytes to the named destination: published. The same again: denied as replay. Both refusals happen before the approval is spent, so neither can be credited to replay protection. An earlier version of our own test got that order wrong.